PES UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

UE24CS341A — Software Engineering | Level 3 Mini-Project

SOFTWARE REQUIREMENTS SPECIFICATION

(SRS)

CampusReady

Online Internship & Placement Preparation Portal

Team Number	Team 12
Project Title	CampusReady – Online Internship & Placement Preparation Portal
Technology Stack	Python / JavaScript — MERN Stack (MongoDB, Express.js, React.js, Node.js)
Complexity	Medium
Stakeholders	College Training & Placement Office (TPO), participating recruiters, and students
Document Version	First Edition

Team Members

Sl.	Student Name	USN
1	Sejal Siddapuram	PES1UG24CS426
2	Saiesh Shankar	PES1UG24CS401
3	Sampath D Naik	PES1UG24CS416
4	Sagar A Walikar	PES1UG24CS396

Table of Contents

1. **Introduction and Purpose**
2. **Product Scope**
3. **Intended Users**
4. **Product Functions**
5. **User Classes and Characteristics**
6. **Operating Environment**
7. **Constraints and Assumptions**
8. **External Interface Requirements**
9. **Use Case / Analysis Models**
10. **Functional Requirements**
11. **Non-Functional Requirements**
12. **Security Requirements**
13. **Business / Domain Rules**
14. **Database / Field Requirements**
15. **Glossary**
16. **Requirement Traceability Matrix**

1 Introduction and Purpose

1.1 Purpose of this Document

This Software Requirements Specification (SRS) describes, in detail, the functional and non-functional requirements for CampusReady – Online Internship & Placement Preparation Portal, developed by Team 12 as part of the UE24CS341A Software Engineering Level 3 mini-project. This document is intended to serve as the single point of reference between the development team, course instructors, and the college Training & Placement Office (TPO) for what the system will and will not do, and forms the baseline against which the system will be designed, built, tested and accepted.

1.2 Document Conventions

- Requirements are uniquely identified using the prefixes FR- (Functional Requirement), NFR- (Non-Functional Requirement), SR- (Security Requirement) and BR- (Business Rule).
- Priority levels used throughout this document are High, Medium and Low, reflecting criticality to the Sprint 1–6 delivery plan.
- The terms “portal”, “system” and “CampusReady” are used interchangeably to refer to the product being specified.

1.3 Project References

- Deliverable 1 – Synopsis / Project Proposal, Team 12, UE24CS341A.
- SRS template prescribed by the Department of Computer Science and Engineering, PES University.

2 Product Scope

Currently, students hear about internships and placements through a mix of noticeboards, generic job sites and word of mouth, and there is no single place to track an application once it has been submitted. CampusReady replaces that with one MERN-stack (MongoDB, Express.js, React.js, Node.js) web application covering internship/placement discovery, application tracking, recruiter management and placement preparation.

In scope for this mini-project:

- Student registration, authentication and profile/resume management.
- Browsing, searching and filtering of internship/job listings.
- Application submission and status tracking (Applied / Shortlisted / Rejected / Selected).
- A recruiter portal to post listings and shortlist applicants.
- A mock aptitude test and interview-preparation practice module.
- A TPO/Admin dashboard for onboarding companies, moderating postings and viewing placement analytics.
- In-app and email notifications for deadlines and status changes.

Out of scope for the current mini-project iteration: payment processing, native mobile applications, video-based interview proctoring, integration with third-party Applicant Tracking Systems (ATS), and multi-college/multi-tenant deployment. These may be considered as future enhancements beyond Sprint 6.

3 Intended Users

- Students – seeking internships/placements, building a profile and tracking applications.
- Recruiters – representing companies that post openings and shortlist candidates.
- TPO / Admin – college staff who onboard companies, moderate content and monitor outcomes.
- Guest / Visitor – an unauthenticated user who can view public information before registering.

4 Product Functions

At a high level, CampusReady provides the following major functions, each elaborated further in Sections 9 and 10:

- Account management: registration, login, password reset, and role-based access for students, recruiters and admins.
- Profile and resume building for students, including academic and skill details.
- Listing management: creation, editing, publishing and search/filtering of internship and job postings.
- Application workflow: submission, withdrawal and real-time status tracking.
- Shortlisting and recruiter-side applicant review.
- Placement preparation: mock aptitude tests and curated interview questions.
- Administrative oversight: company onboarding, posting moderation, and analytics/reporting.
- Notifications: in-app and email alerts for deadlines and status changes.

5 User Classes and Characteristics

User Class	Description	Key Privileges
Student	Registered candidate seeking internships/placements. Assumed to have basic web literacy and college-issued email/SRN.	Manage own profile; browse/apply to listings; track application status; attempt mock tests
Recruiter	Company representative onboarded by the TPO to post and manage openings.	Post/edit listings; view applicants; shortlist/reject candidates
TPO / Admin	College Training & Placement Office staff with the highest level of system access.	Onboard companies; moderate/approve postings; manage users; view analytics
Guest	Unauthenticated visitor.	View public landing pages; register for an account

6 Operating Environment

Layer	Detail
Client	Modern evergreen web browsers (Chrome, Edge, Firefox, Safari) on desktop and mobile viewports; responsive React.js single-page application
Front End	React.js (JavaScript), HTML5, CSS3
Back End	Node.js with Express.js exposing a RESTful API

Layer	Detail
Database	MongoDB (NoSQL document store)
Hosting / Deployment	Cloud or campus-hosted application server; environment-based configuration for development, staging and production
Authentication	Token-based session management (e.g., JWT) over HTTPS

7 Constraints and Assumptions

7.1 Constraints

- The project must be delivered using the Python/JavaScript MERN stack as fixed in the approved synopsis.
- Development is time-boxed to six sprints (Sprint 1–6) within a single academic semester, as per the Short-Term Plan.
- The team consists of four student developers with shared responsibility across design, development and testing.
- The system is designed for a single-college deployment and is not required to support multi-tenancy in this iteration.

7.2 Assumptions

- Students and recruiters have access to a reasonably reliable internet connection and a supported browser.
- The TPO will provide/approve the initial list of companies permitted to be onboarded as recruiters.
- Email delivery for notifications relies on a standard SMTP or transactional email service being available in the deployment environment.
- Data volumes (students, listings, applications) remain within the scale of a single college and do not require distributed database sharding.

8 External Interface Requirements

8.1 User Interfaces

- A responsive React.js web interface with distinct views/dashboards for Student, Recruiter and Admin roles.
- Consistent navigation, form validation feedback, and accessible colour contrast across all pages.

8.2 Hardware Interfaces

- None beyond a standard client device (desktop, laptop or mobile) capable of running a modern web browser.

8.3 Software Interfaces

- MongoDB database accessed via the Node.js/Express.js backend using an ODM such as Mongoose.
- SMTP/email service provider interface for outbound notification emails.
- RESTful JSON API between the React.js front end and the Express.js back end.

8.4 Communication Interfaces

- All client-server communication occurs over HTTPS.
- In-app notifications are delivered over the existing REST/WebSocket-style polling channel; email notifications are delivered via SMTP.

9 . Use Case / Analysis Models

The primary actors of CampusReady are Student, Recruiter and TPO/Admin, each interacting with the system through role-specific use cases as summarised below. Guest is a limited actor able only to register or view public content.

Use Case ID	Name	Primary Actor	Description
UC-01	Register / Login	Student, Recruiter, Admin	Create an account or authenticate to access role-specific features
UC-02	Manage Profile	Student	Create and update academic, personal and resume details
UC-03	Search & Filter Listings	Student	Browse internship/job postings using keyword, role, company or location filters
UC-04	Apply to Listing	Student	Submit an application to a chosen listing and track its status
UC-05	Post Listing	Recruiter	Create and publish a new internship/job opening
UC-06	Shortlist Applicants	Recruiter	Review submitted applications and update their status
UC-07	Take Mock Test	Student	Attempt an aptitude test or view interview-preparation content
UC-08	Onboard Company	Admin	Register a new recruiter/company on the platform
UC-09	Moderate Postings	Admin	Approve, edit or remove listings that violate platform policy
UC-10	View Analytics	Admin	View placement statistics and operational reports
UC-11	Receive Notification	Student, Recruiter	Receive in-app/email alerts for deadlines and status changes

9.1 Sample Use Case Description — UC-04 Apply to Listing

Field	Description
Actor	Student
Precondition	Student is registered, logged in, and has a complete profile
Main Flow	1. Student searches/filters listings. 2. Student opens a listing detail page. 3. Student clicks Apply. 4. System attaches the student's profile/resume to the application. 5. System sets status to Applied and notifies the student.
Alternate Flow	If the profile is incomplete, the system prompts the student to complete required fields before allowing submission.

Field	Description
Postcondition	An application record is created and visible to both the student and the owning recruiter

9.2 Use Case Diagram

Figure 1 shows the actors of CampusReady and the use cases listed above. Student, Recruiter and TPO/Admin use the system directly, while a Guest can only register. The email (SMTP) service is an external actor that delivers notifications. Applying to a listing extends searching, since a student applies from a listing found through search.

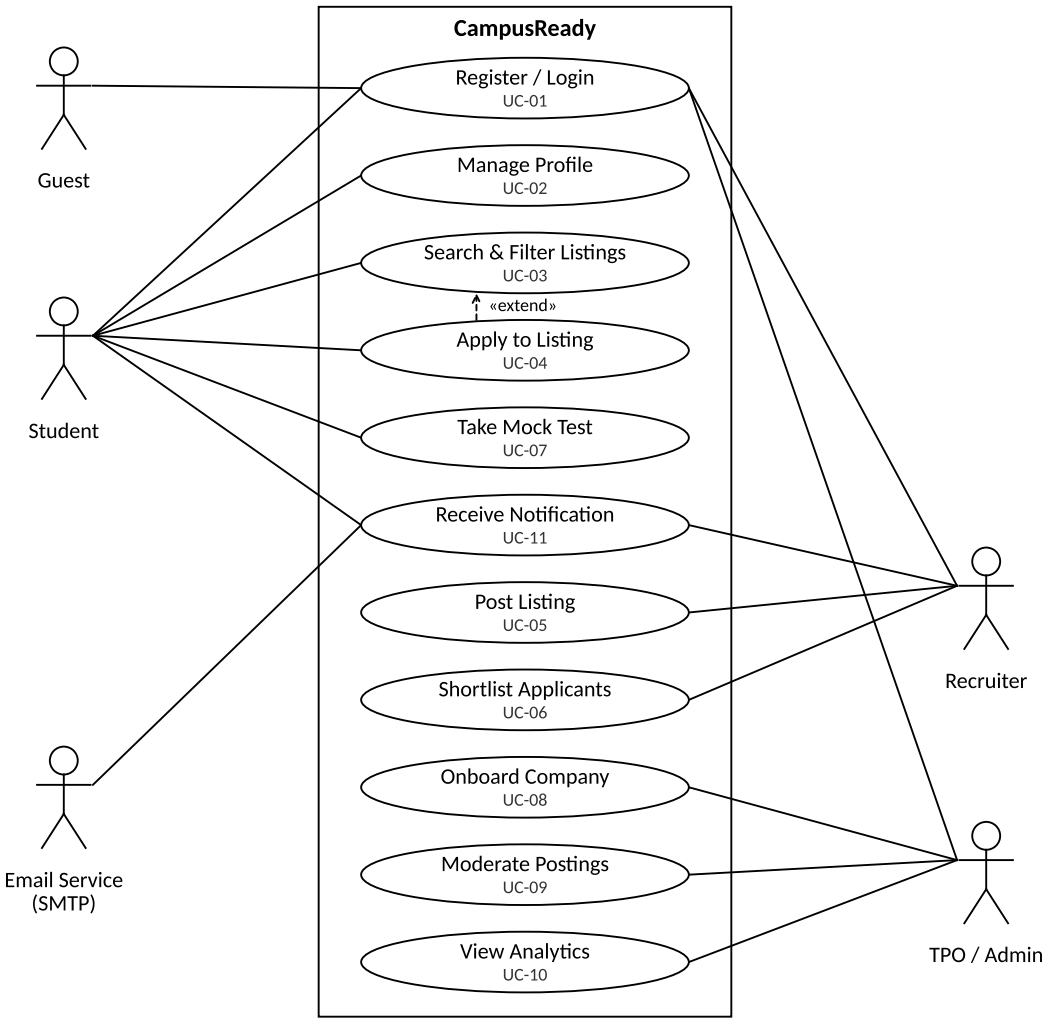


Figure 1: Use case diagram for CampusReady

10 Functional Requirements

10.1 Account & Profile Management — Owner: Sejal Siddapuram

ID	Requirement	Priority
FR-01	The system shall allow a new student to register using college-issued credentials (name, SRN/USN, email, password).	High
FR-02	The system shall authenticate registered users via login with email and password, and enforce role-based access control distinguishing Student, Recruiter and Admin capabilities.	High
FR-03	The system shall allow students to create and edit a profile including academic details, skills and an uploadable resume.	High

10.2 Listings & Applications — Owner: Sampath D Naik

ID	Requirement	Priority
FR-04	The system shall allow students to search and filter listings by keyword, role, company, location and deadline.	High
FR-05	The system shall allow a student to submit an application to a listing, attaching their current profile/resume.	High
FR-06	The system shall track and display application status as Applied, Shortlisted, Rejected or Selected, allow a student to withdraw an application prior to a recruiter decision, and notify the student when the status changes.	High

10.3 Recruiter Portal — Owner: Sagar A Walikar

ID	Requirement	Priority
FR-07	The system shall allow recruiters to create, edit and publish internship/job listings with title, description, eligibility and deadline.	High
FR-08	The system shall allow recruiters to view a list of applicants for each of their listings and update applicant status.	High
FR-09	The system shall provide recruiters with a summary count of applicants by status for each of their listings.	Medium

10.4 Admin & Preparation — Owner: Saiesh Shankar

ID	Requirement	Priority
FR-10	The system shall allow the Admin/TPO to onboard new companies/recruiter accounts and to moderate (approve, edit, or remove) postings before or after publication.	High
FR-11	The system shall provide the Admin/TPO with a dashboard summarising operational and placement statistics.	Medium
FR-12	The system shall provide a mock aptitude test module with objective questions and automatic scoring, plus curated interview-preparation content accessible to students.	Medium

11 Non-Functional Requirements

Category	Requirement
Performance	NFR-01: Core pages (listing search, dashboard) shall load within 3 seconds under normal load on a broadband connection.
Usability	NFR-02: The interface shall follow a consistent design language and be usable by a first-time student without training, on both desktop and mobile viewports.
Reliability	NFR-03: The system shall correctly persist application status transitions with no data loss during normal operation.
Availability	NFR-04: The portal shall target availability during college working hours consistent with a single-server academic deployment (best-effort outside a formal SLA).
Maintainability	NFR-05: The codebase shall follow a modular MVC-style structure (Express routes/controllers, Mongoose models, React components) to ease future enhancement.
Scalability	NFR-06: The data model and API shall be able to accommodate growth in the number of students, listings and applications for a single-college scale without redesign.
Portability	NFR-07: The front end shall render correctly on the latest two major versions of Chrome, Edge and Firefox.

12 Security Requirements

ID	Requirement
SR-01	Passwords shall be stored using a salted one-way hash (e.g., bcrypt); plaintext passwords shall never be persisted or logged.
SR-02	All client-server traffic shall be encrypted in transit using HTTPS.
SR-03	Access to Recruiter and Admin functionality shall be enforced server-side via role-based authorization checks on every protected API route, not only hidden in the UI.
SR-04	Session/authentication tokens shall have a defined expiry and be invalidated on logout.
SR-05	Uploaded resume files shall be validated for file type and size before storage to reduce the risk of malicious uploads.
SR-06	The system shall log security-relevant events (failed logins, admin actions) for audit purposes.

13 Business / Domain Rules

ID	Rule
BR-01	A student may hold at most one active application per listing; re-application requires prior withdrawal.
BR-02	Only listings approved by the Admin/TPO (directly or via pre-approved recruiter accounts) are visible to students.
BR-03	An application can only move to Shortlisted, Rejected or Selected by the recruiter who owns the corresponding listing.
BR-04	A listing past its stated deadline shall no longer accept new applications but remains visible for historical/status purposes.

ID	Rule
BR-05	Only the Admin/TPO role may onboard new recruiter/company accounts.

14 Database / Field Requirements

14.1 User (Base)

Field	Type	Constraints
_id	ObjectId	Primary key
name	String	Required
email	String	Required, unique
passwordHash	String	Required
role	Enum (Student, Recruiter, Admin)	Required
createdAt	Date	Auto-generated

14.2 Student Profile

Field	Type	Constraints
userId	ObjectId (ref: User)	Required
srn / usn	String	Required, unique
branch, semester	String / Number	Required
skills	Array<String>	Optional
resumeUrl	String	Optional

14.3 Listing

Field	Type	Constraints
_id	ObjectId	Primary key
recruiterId	ObjectId (ref: User)	Required
title, description	String	Required
eligibility	String	Optional
deadline	Date	Required
status	Enum (Draft, Published, Closed)	Required, default Draft

14.4 Application

Field	Type	Constraints
_id	ObjectId	Primary key
studentId	ObjectId (ref: User)	Required
listingId	ObjectId (ref: Listing)	Required

Field	Type	Constraints
status	Enum (Applied, Shortlisted, Rejected, Selected)	Required, default Applied
appliedAt	Date	Auto-generated

15 Glossary

Term	Definition
TPO	Training & Placement Office — the college department coordinating internships and placements
MERN Stack	MongoDB, Express.js, React.js, Node.js — the JavaScript technology stack used to build the portal
Recruiter	A company representative authorised to post and manage listings on the portal
Listing	An internship or job opening posted by a recruiter
SRN / USN	Student Registration Number / University Seat Number, the unique student identifiers used at PES University
JWT	JSON Web Token — a compact token format commonly used for stateless authentication

16 Requirement Traceability Matrix

Req ID	Source Objective / Feature	Delivered In	Verification
FR-01, FR-02, FR-03	Student registration, login, RBAC & profile management (Sejal Siddapuram)	Sprint 1–2	Working login demo, unit tests
FR-04	Listing search & filtering (Sampath D Naik)	Sprint 2	Demo + unit tests
FR-05, FR-06	Application submission, status tracking & withdrawal (Sampath D Naik)	Sprint 3	Integration tests + PRs
FR-07	Recruiter portal: post listings (Sagar A Walikar)	Sprint 4	System demo + test evidence
FR-08, FR-09	Recruiter portal: view/shortlist applicants & applicant summary (Sagar A Walikar)	Sprint 4	System demo + test evidence
FR-10, FR-11	Admin/TPO onboarding, moderation & dashboard (Saiesh Shankar)	Sprint 5	Validation + coverage report
FR-12	Mock aptitude test & interview-prep module (Saiesh Shankar)	Sprint 5	Validation + coverage report
SR-01 to SR-06	Security hardening	Sprint 5	Validation + coverage report
NFR-01 to NFR-07	Non-functional quality attributes (performance, usability, reliability, availability, maintainability, scalability, portability)	Sprint 6	Performance/load testing, usability walkthroughs, cross-browser checks, code review
BR-01 to BR-05	Business/domain rules	Sprints 1–5	Verified alongside the owning functional

Req ID	Source Objective / Feature	Delivered In	Verification
			requirement via unit/integration tests
All	System validation and release	Sprint 6	Final report + deployment/demo

PES UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

UE24CS341A, Software Engineering | Level 3 Mini-Project

SOFTWARE ARCHITECTURE AND DESIGN SPECIFICATION

(SAD)

CampusReady: Online Internship and Placement Preparation Portal

Project	CampusReady: Online Internship and Placement Preparation Portal
Team Number	Team 12
Version	1.0
Authors	Sejal Siddapuram, Sampath D Naik, Sagar A Walikar, Saiesh Shankar
Date	01-10-2026
Status	Draft

Team Members

Sl.	Student Name	USN	Module Owned
1	Sejal Siddapuram	PES1UG24CS426	Account and Profile Management (FR-01 to FR-03)
2	Sampath D Naik	PES1UG24CS416	Listings and Applications (FR-04 to FR-06)
3	Sagar A Walikar	PES1UG24CS396	Recruiter Portal (FR-07 to FR-09)
4	Saiesh Shankar	PES1UG24CS401	Admin and Preparation (FR-10 to FR-12)

Revision History

Version	Date	Author	Change Summary
1.0	01-10-2026	Team 12	First edition of the architecture and design specification, based on SRS First Edition.

Approvals

Role	Name	Signature / Date
Course Instructor		
Team Representative		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of CampusReady, the Online Internship and Placement Preparation Portal developed by Team 12. It describes how the requirements in the CampusReady SRS (First Edition) are realised as components, interfaces, data structures and interactions, and it forms the basis for implementation in Sprints 1 to 6 and for the updated Software Test Plan.

1.2 Scope

The document covers all twelve functional requirements (FR-01 to FR-12), the non-functional requirements (NFR-01 to NFR-07), the security requirements (SR-01 to SR-06) and the business rules (BR-01 to BR-05) of the SRS. It covers account and profile management, listing search and posting, application submission and tracking, the recruiter portal, the admin dashboard, the mock test module and notifications. Items that the SRS places out of scope (payments, native mobile applications, video interview proctoring, third-party ATS integration and multi-tenant deployment) are not designed here.

1.3 Audience

Developers of Team 12, the course instructors who evaluate the design, QA and security reviewers, and the college Training and Placement Office (TPO) as the customer representative.

1.4 Definitions

SPA (single page application), REST (representational state transfer), JWT (JSON Web Token), RBAC (role-based access control), ODM (object document mapper), TLS (transport layer security), STRIDE (Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege), SRS (Software Requirements Specification), STP (Software Test Plan), RTM (Requirement Traceability Matrix). Further terms are listed in Section 5.1.

2. Document Overview

2.1 How to use this document

Section 3 presents the architecture: goals, the component diagram, the chosen pattern, technology stack, risks, traceability to requirements and the security architecture. Section 4 presents the design: module and data design, UML sequence diagrams for three flows, the REST API definition, error handling, logging and monitoring, and UX design. Section 5 holds the glossary, references and tools.

2.2 Related Documents

CampusReady Synopsis (Deliverable 1), CampusReady SRS First Edition (Deliverable 2), and CampusReady Software Test Plan Version 1.0, which is submitted together with this document. The RTM in Section 3.8 extends Section 16 of the SRS with architecture, design and test references.

3. Architecture

3.1 Goals and Constraints

Goals. (1) Provide one secure web portal for students, recruiters and the TPO. (2) Keep the code modular so that four team members can own separate functional modules and merge work with few conflicts (NFR-05). (3) Enforce role-based access and business rules on the server for every protected request (SR-03, BR-01 to BR-05). (4) Keep core pages within 3 seconds under normal load (NFR-01). (5) Allow the data model and API to grow to the scale of a single college without redesign (NFR-06).

Constraints. MERN stack fixed by the approved synopsis; six sprints in one semester; four developers; single-college deployment without multi-tenancy; HTTPS for all traffic; Agile process with CI/CD, code review and security validation as mandatory practices of the course.

3.2 Stakeholders and Concerns

Stakeholder	Main concerns	Architectural response
Students	Easy discovery, reliable application tracking, privacy of profile and resume	Responsive SPA, persisted status transitions, owner-only access to own data
Recruiters	Control over own listings and applicants only	Server-side RBAC and ownership checks (BR-03)
TPO / Admin	Onboarding control, moderation, analytics, audit trail	Admin-only routes (BR-05), audit logging, aggregation queries
Developers	Modularity, testability, parallel work	Layered structure with one service per module and a separate data access layer
Instructors / reviewers	Traceability, security validation, explainable design	Requirement to component to API to test mapping (Section 3.8)

3.3 Component (UML) Diagram

Figure 1 shows the components of CampusReady grouped by layer, with the external systems they depend on. Requirement tags in the service boxes show which functional requirements each service realises.

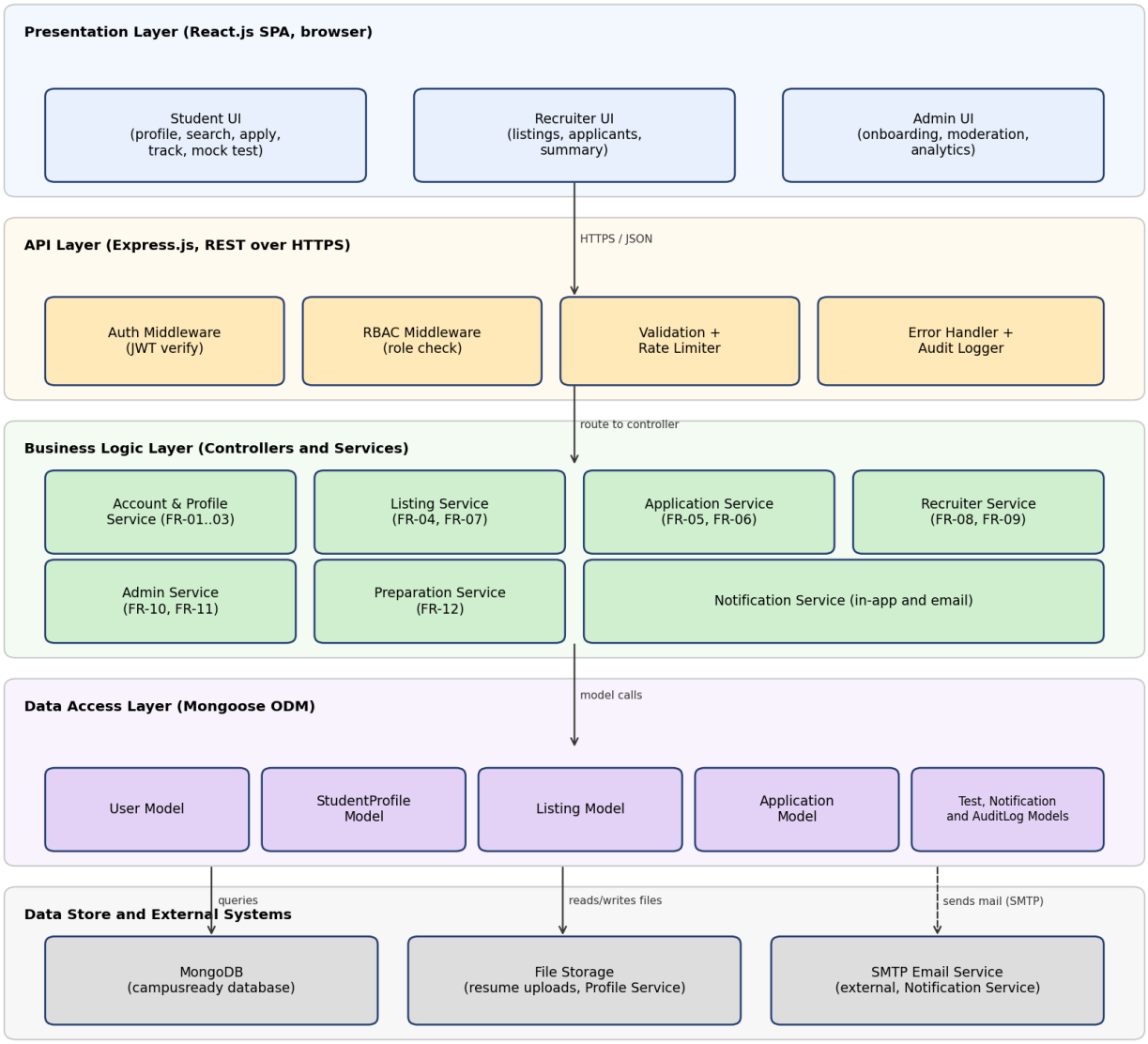


Figure 1: CampusReady component diagram (layers, services and external systems)

3.4 Component Descriptions

Component	Responsibility	Requirements
Student UI, Recruiter UI, Admin UI	React.js views and role-specific dashboards. Perform client-side form validation, hold the JWT in memory or session storage, call the REST API over HTTPS and render errors using the standard error format.	NFR-02, NFR-07
Auth Middleware	Extracts the bearer token, verifies signature and expiry, rejects tokens present in the revoked token list, and attaches user id and role to the request.	SR-04, FR-02
RBAC Middleware	Allows a route only for the roles declared on it (Student, Recruiter, Admin). Applied on every protected route.	SR-03, FR-02
Validation and Rate Limiter	Schema validation and sanitisation of body, query and parameters; request throttling on login and registration; blocks operator injection keys in input.	SR-05, Security
Error Handler and Audit Logger	Converts all errors to the standard error format, hides stack traces from clients, and writes security events (failed logins, admin actions) to the audit log.	SR-06
Account and Profile Service	Registration, password hashing, login, logout, token issue and revocation, student profile and resume upload handling.	FR-01, FR-02, FR-03, SR-01, SR-04, SR-05
Listing Service	Create, edit, publish and close listings; search and filter of published listings with pagination.	FR-04, FR-07, BR-02, BR-04
Application Service	Submit and withdraw applications, enforce one active application per listing and the deadline rule, status transitions and student application history.	FR-05, FR-06, BR-01, BR-03, BR-04
Recruiter Service	Applicant lists per owned listing, status updates by the owning recruiter, and per-listing status counts.	FR-08, FR-09, BR-03
Admin Service	Recruiter and company onboarding, listing moderation, and the analytics aggregation for the dashboard.	FR-10, FR-11, BR-05
Preparation Service	Mock aptitude tests with objective questions, automatic scoring, storage of attempts, and curated interview content.	FR-12
Notification Service	Creates in-app notifications and sends email through SMTP for status changes and approaching deadlines. Email failure does not fail the triggering request.	UC-11, FR-06
Mongoose Models	User, StudentProfile, Listing, Application, TestQuestion, TestAttempt, Notification, AuditLog and RevokedToken schemas with validation and indexes.	SRS Section 14, NFR-03, NFR-06
MongoDB	Primary document store for all application data.	All
File Storage	Stores validated resume files outside the web root; the database stores only the file reference.	FR-03, SR-05
SMTP Email Service	External service used for outbound notification email.	UC-11

3.5 Chosen Architecture Pattern and Rationale

Chosen pattern: layered client-server architecture with an MVC-style separation on the server. The React single page application is the client. The Express.js server is organised into API layer (routes and middleware), business logic layer (controllers and services) and data access layer (Mongoose models). Communication between client and server is stateless REST over HTTPS with JSON bodies.

Rationale.

- Separation of concerns: each layer can be tested on its own, for example services with mocked models and routes with a test client (NFR-05).
- Parallel ownership: each team member owns one service and its routes, controllers and models, which keeps merge conflicts low and gives each member one complete functional module.
- Security: all cross-cutting controls (authentication, RBAC, validation, rate limiting, logging) sit in one middleware chain that every request must pass, so no module can bypass them (SR-03).
- Fit with scale: the system serves a single college, so one deployable unit is sufficient (NFR-06).

Alternatives considered and rejected. A microservices architecture was rejected because it adds deployment, networking and distributed-data complexity that is not justified for a single-college system built by four students

in six sprints. A server-rendered monolith without a separate SPA was rejected because the synopsis fixes React.js and the role-specific dashboards benefit from a client-side application.

3.6 Technology Stack and Data Stores

Concern	Choice
Front end	React.js, HTML5, CSS3, React Router, Axios
Back end	Node.js, Express.js
Database	MongoDB accessed through Mongoose ODM
Authentication	JWT with fixed expiry; passwords hashed with bcrypt (salted)
Security libraries	helmet (HTTP headers), express-rate-limit, express-validator, express-mongo-sanitize, cors with an allow-list
File upload	multer with file type and size checks; files stored outside the public directory
Email	Nodemailer over SMTP
Transport	HTTPS (TLS 1.2 or higher)
Testing and quality	Jest, Supertest, React Testing Library, Postman, SonarQube or similar static analysis
Build and delivery	Git and GitHub, Jenkins for CI/CD, Docker for containerisation
Data stores	MongoDB collections: users, studentprofiles, listings, applications, testquestions, testattempts, notifications, auditlogs, revokedtokens (TTL index). Resume files in server-side file storage.

3.7 Risks and Mitigations

Risk	Mitigation
Role checks implemented only in the UI	Mandatory RBAC middleware on every protected route; security tests call the API directly with the wrong role (SR-03)
Slow search as listings grow	Indexes on listing status, deadline, title and company; pagination with a page size limit (NFR-01, NFR-06)
Malicious or oversized resume upload	Allow-list of file type, size limit, renamed stored file, storage outside web root (SR-05)
Email service unavailable	Notification sending is asynchronous and non-blocking; the in-app notification is always stored; failures are logged
Team members changing shared contracts	API contracts in Section 4.3 are the baseline; changes need a pull request with reviewer approval
Token theft or reuse after logout	Short token expiry, revoked token list with TTL index, HTTPS only (SR-02, SR-04)
Data loss in status transitions	Status updates are single atomic document updates with schema validation (NFR-03)

3.8 Traceability to Requirements

The table maps each SRS requirement to the architectural component that realises it and to the primary API endpoints defined in Section 4.3. Test case identifiers are given in the Software Test Plan, Section 13.

Req ID	Component(s)	API / Design reference
FR-01	Account and Profile Service, Validation	POST /api/auth/register; Sequence Diagram 1
FR-02	Auth Middleware, RBAC Middleware, Account Service	POST /api/auth/login, POST /api/auth/logout; Sequence Diagram 1
FR-03	Account and Profile Service, File Storage	GET and PUT /api/profile, POST /api/profile/resume
FR-04	Listing Service	GET /api/listings, GET /api/listings/:id
FR-05	Application Service	POST /api/applications; Sequence Diagram 2
FR-06	Application Service, Notification Service	GET /api/applications/me, DELETE /api/applications/:id, GET /api/notifications
FR-07	Listing Service	POST /api/listings, PUT /api/listings/:id, PATCH /api/listings/:id/publish

Req ID	Component(s)	API / Design reference
FR-08	Recruiter Service, Application Service	GET /api/recruiter/listings/:id/applications, PATCH /api/applications/:id/status; Sequence Diagram 3
FR-09	Recruiter Service	GET /api/recruiter/listings/:id/summary
FR-10	Admin Service	POST /api/admin/recruiters, PATCH /api/admin/listings/:id/moderation
FR-11	Admin Service	GET /api/admin/analytics
FR-12	Preparation Service	GET /api/prep/tests, POST /api/prep/tests/:id/submit, GET /api/prep/interview
NFR-01, NFR-06	Listing Service, MongoDB indexes, pagination	Section 3.7, Section 4.1 (indexes)
NFR-02, NFR-07	Student, Recruiter and Admin UI	Section 4.5
NFR-03	Application Model, atomic updates	Section 4.1
NFR-04	Deployment on a single server with a health endpoint	GET /api/health; Section 4.4
NFR-05	Layered structure of Section 3.5	Folder structure in Section 4.1
SR-01	Account and Profile Service (bcrypt)	Section 3.9
SR-02	HTTPS termination, helmet HSTS	Section 3.9
SR-03	RBAC Middleware	Section 3.9; every protected route in Section 4.3
SR-04	Auth Middleware, RevokedToken collection	POST /api/auth/logout; Sequence Diagram 1
SR-05	File upload validation	POST /api/profile/resume
SR-06	Audit Logger, AuditLog collection	Section 4.4
BR-01, BR-04	Application Service	POST /api/applications; Sequence Diagram 2
BR-02	Listing Service, Admin Service	GET /api/listings (Published and approved only)
BR-03	Application Service, Recruiter Service	PATCH /api/applications/:id/status; Sequence Diagram 3
BR-05	Admin Service, RBAC Middleware	POST /api/admin/recruiters

3.9 Security Architecture

Security objectives

- **Confidentiality:** only the owner of an account can read its profile, resume and applications; recruiters see only applicants of their own listings.
- **Integrity:** application status and listing data change only through authorised, validated requests that follow the business rules.
- **Availability:** abusive request patterns such as login brute force are throttled so that normal users are not affected.
- **Accountability:** security-relevant events are logged so that actions can be traced to an account.

Trust boundaries and controls

There are two trust boundaries. The first lies between the browser and the Express.js server; all input crossing it is untrusted and passes the middleware chain of authentication, RBAC, validation and rate limiting. The second lies between the server and external systems (MongoDB, file storage, SMTP); credentials for these are held in environment variables and never in source control.

Threat model (STRIDE)

Threat	Example in CampusReady	Control	SR
Spoofing	Attacker logs in as another user or reuses a stolen token	bcrypt-verified login, signed JWT with expiry, revoked token list, login rate limiting, HTTPS	SR-01, SR-02, SR-04

Threat	Example in CampusReady	Control	SR
Tampering	Student modifies another student's application or a recruiter edits another recruiter's listing; NoSQL operator injection in request body	Server-side ownership checks, schema validation, input sanitisation that strips operator keys	SR-03
Repudiation	Admin denies removing a posting	Audit log of admin actions and failed logins with user id, action, time and source address	SR-06
Information disclosure	Plaintext password in database or logs; resume of one student exposed to another; stack trace shown to client	Hashing, no secrets in logs, owner-only resume access, generic error responses, helmet headers	SR-01, SR-03
Denial of service	Brute force on login, flooding of search, very large uploads	express-rate-limit, pagination limits, upload size limit, request body size limit	SR-05
Elevation of privilege	Student calls an admin or recruiter endpoint directly	RBAC middleware on every protected route; role stored in the signed token and re-checked against the user record for admin routes	SR-03

Role access matrix

Capability	Guest	Student	Recruiter	Admin
Register, login	Yes	Yes	Yes	Yes
View and search published listings	Public view only	Yes	Own listings	All
Manage own profile and resume	No	Yes	No	No
Apply, withdraw, track applications	No	Yes	No	No
Create and publish listings	No	No	Yes (own)	No
View applicants, update status	No	No	Yes (own listings)	No
Onboard recruiters, moderate, analytics	No	No	No	Yes
Mock tests and interview content	No	Yes	No	Manage content

4. Design

4.1 Design Overview

The server is implemented as a set of feature modules. Each module contains a route file, a controller, a service and, where needed, a Mongoose model. Controllers handle HTTP concerns only; services hold business rules and are the unit of testing. Models are accessed only from services.

Module (owner)	Contents	Requirements
Account and Profile (Sejal Siddapuram)	auth routes, profile routes, Account Service, User and StudentProfile models, token revocation, resume upload	FR-01 to FR-03, SR-01, SR-04, SR-05
Listings and Applications (Sampath D Naik)	listing search routes, application routes, Application Service, Application model, student notifications on status change	FR-04 to FR-06, BR-01, BR-04
Recruiter Portal (Sagar A Walikar)	listing management routes, applicant routes, Recruiter Service, Listing model	FR-07 to FR-09, BR-03
Admin and Preparation (Saiesh Shankar)	admin routes, Admin Service, Preparation Service, TestQuestion and TestAttempt models, AuditLog	FR-10 to FR-12, BR-05, SR-06
Shared (all, reviewed by all)	middleware chain, error handler, Notification Service, configuration, CI pipeline	SR-02, SR-03, NFR-05

Proposed server folder structure: `server/src/{routes, controllers, services, models, middleware, utils, config}` and `client/src/{pages, components, services, context}`.

Data design. The entities of SRS Section 14 (User, StudentProfile, Listing, Application) are implemented as specified. The design adds these collections: **TestQuestion** (testId, text, options, correctIndex, topic), **TestAttempt** (studentId, testId, score, takenAt), **Notification** (userId, message, type, read, createdAt), **AuditLog** (userId, action, outcome, ip, createdAt) and **RevokedToken** (tokenId, expiresAt, with a TTL index so

that entries are removed after the token would have expired).

Indexes. users.email (unique); studentprofiles.usn (unique); applications on (studentId, listingId) unique where the application is active, which enforces BR-01 at database level; listings on (status, deadline) and a text index on title, description and company for search (NFR-01).

Application status and withdrawal. Application.status takes the values Applied, Shortlisted, Rejected and Selected. A withdrawn application is removed, so that the student may re-apply as BR-01 allows.

4.2 UML Sequence Diagrams

Three flows specific to CampusReady are modelled. Sequence Diagram 1 covers authentication, Sequence Diagram 2 covers the main student journey, and Sequence Diagram 3 covers the recruiter decision that triggers a notification.

Sequence Diagram 1: Register / Login (UC-01, FR-01, FR-02, SR-01, SR-04)

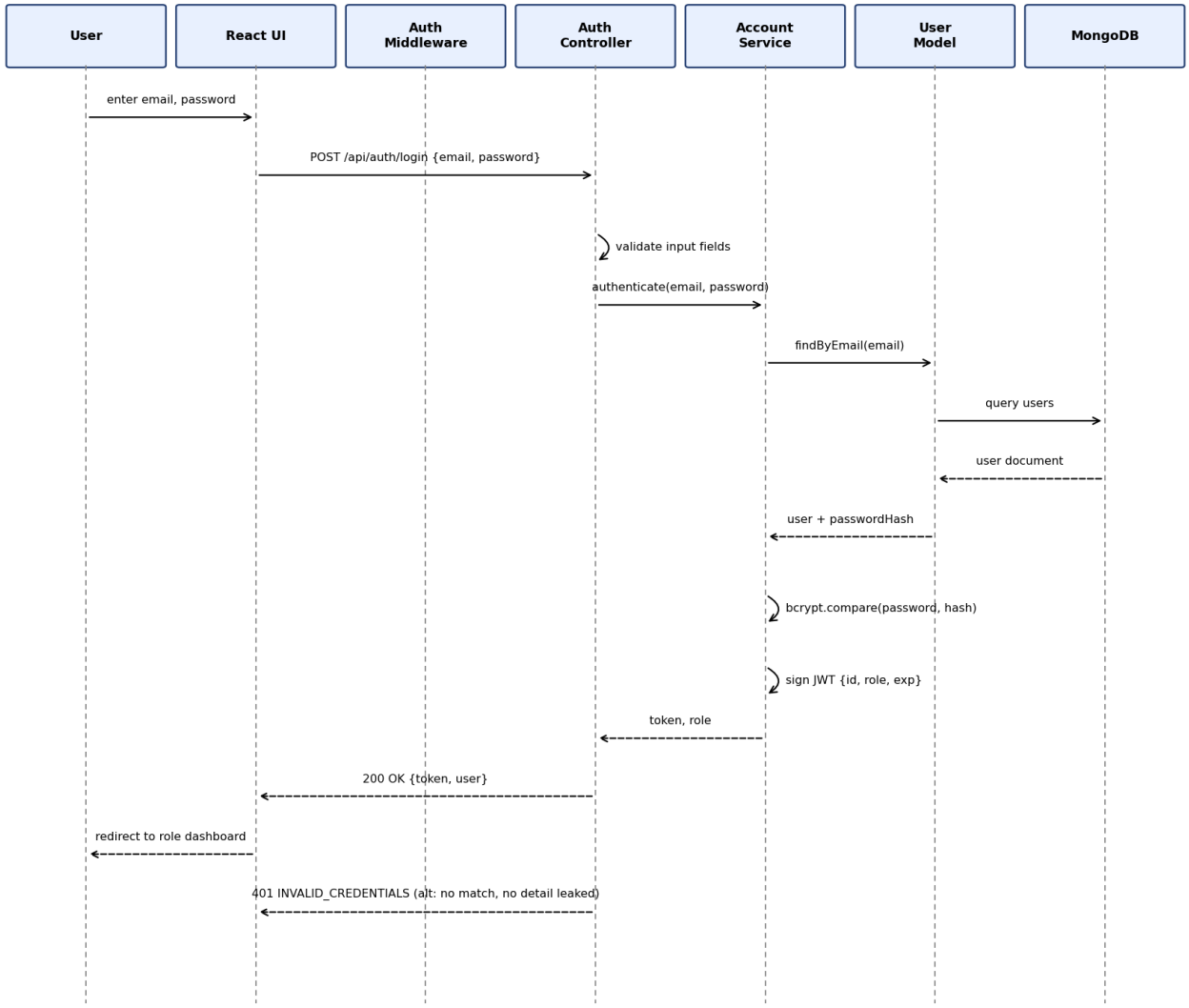


Figure 2: Sequence diagram for Register / Login

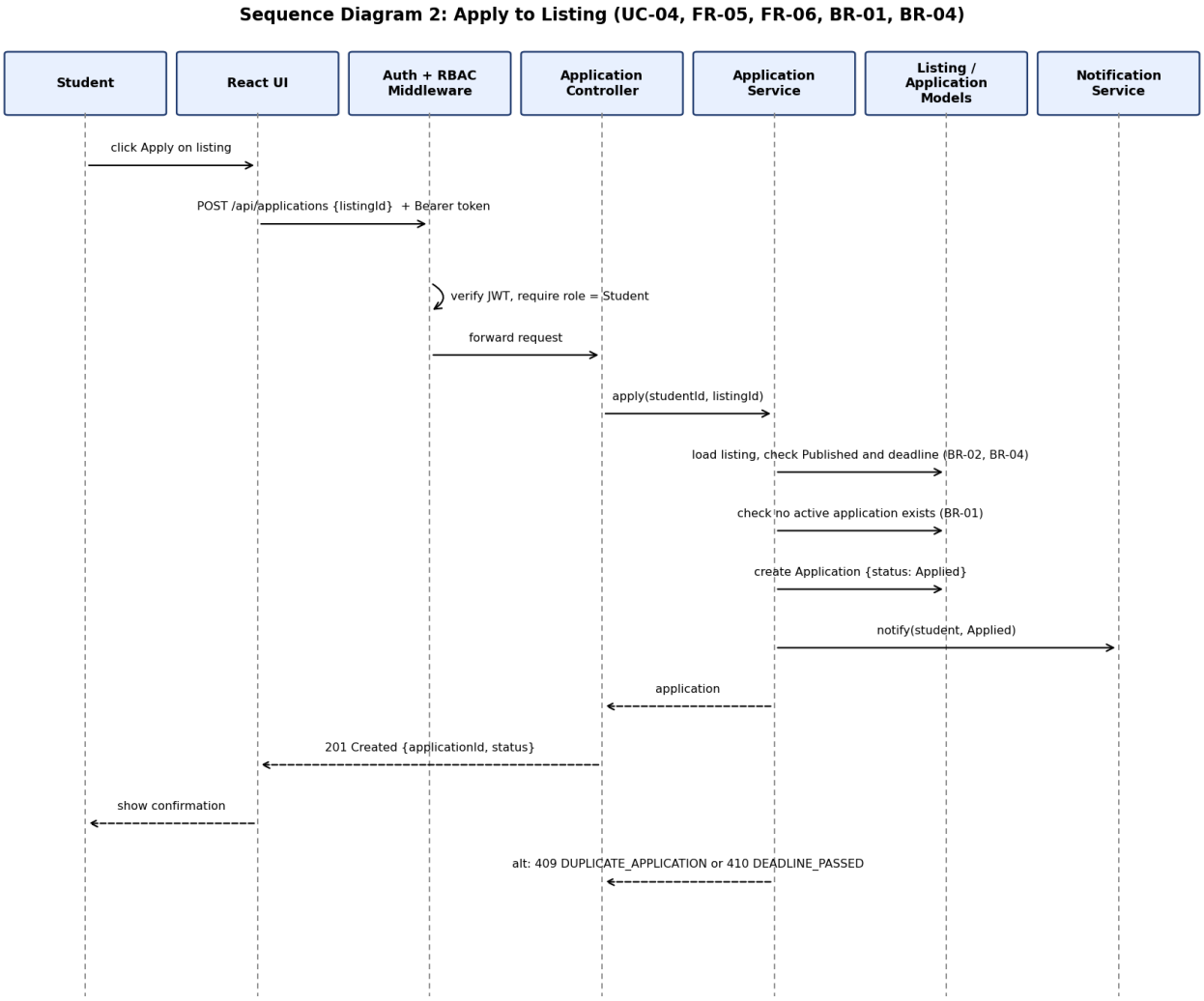


Figure 3: Sequence diagram for Apply to Listing

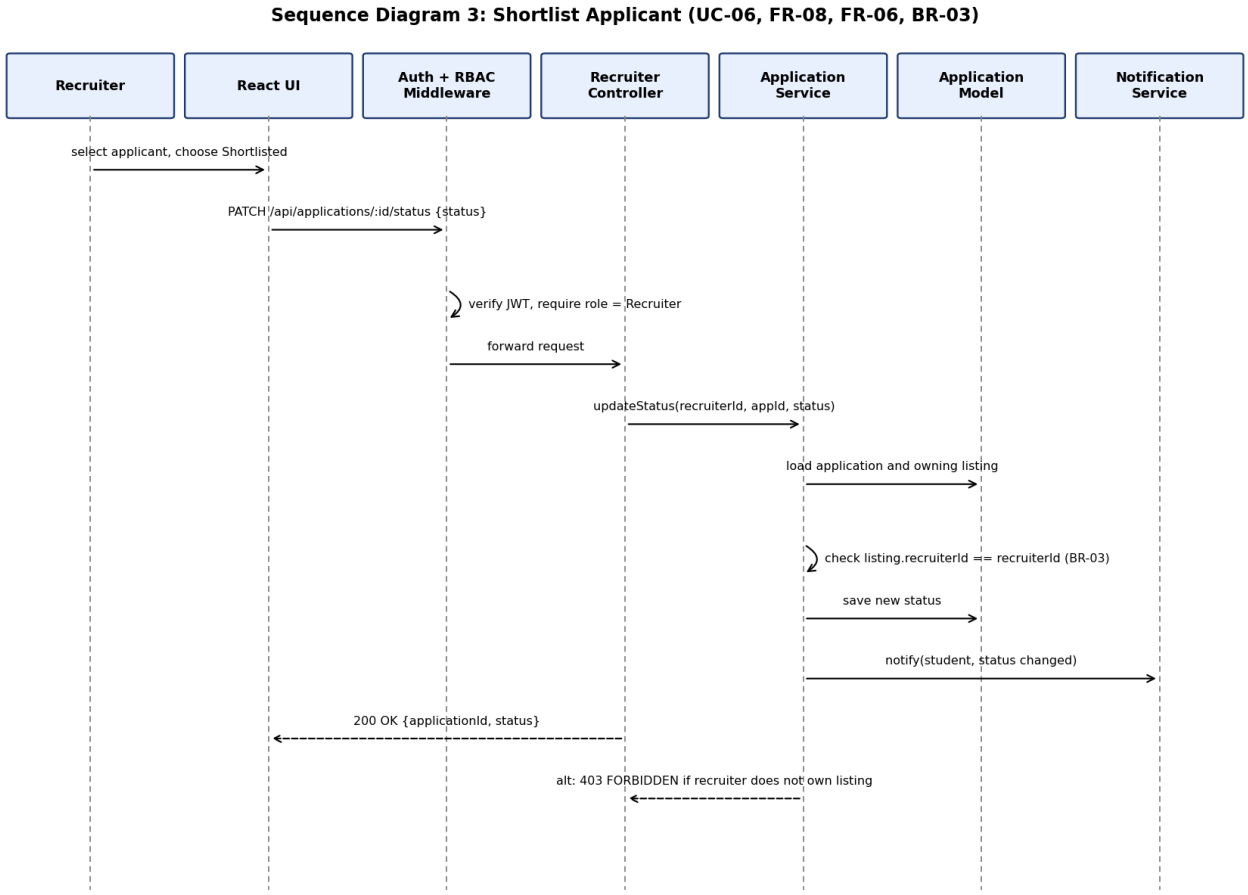


Figure 4: Sequence diagram for Shortlist Applicant

4.3 API Design

Conventions. Base path /api. JSON request and response bodies. Authentication by header `Authorization: Bearer <token>`. Lists are paginated with `page` and `limit` (maximum 50). Identifiers are MongoDB ObjectIds as strings. Success responses use the form `{success: true, data: ...}`; error responses follow Section 4.4. The contracts below are the baseline; changes require a reviewed pull request.

Endpoint summary

Method	Endpoint	Role	Purpose	Req
POST	/api/auth/register	Public	Register a student account	FR-01
POST	/api/auth/login	Public	Authenticate and obtain token	FR-02
POST	/api/auth/logout	Any user	Revoke the current token	SR-04
GET / PUT	/api/profile	Student	Read or update own profile	FR-03
POST	/api/profile/resume	Student	Upload resume (PDF, up to 2 MB)	FR-03, SR-05
GET	/api/listings	Student	Search and filter published listings	FR-04
GET	/api/listings/:id	Student	Listing detail	FR-04
POST	/api/listings	Recruiter	Create a listing (Draft)	FR-07
PUT	/api/listings/:id	Recruiter (owner)	Edit a listing	FR-07
PATCH	/api/listings/:id/publish	Recruiter (owner)	Publish or close a listing	FR-07
POST	/api/applications	Student	Apply to a listing	FR-05
GET	/api/applications/me	Student	List own applications with status	FR-06
DELETE	/api/applications/:id	Student (owner)	Withdraw before a decision	FR-06

Method	Endpoint	Role	Purpose	Req
PATCH	/api/applications/:id/status	Recruiter (owner of listing)	Set Shortlisted, Rejected or Selected	FR-08, BR-03
GET	/api/recruiter/listings/:id/applications	Recruiter (owner)	Applicants of a listing	FR-08
GET	/api/recruiter/listings/:id/summary	Recruiter (owner)	Applicant counts by status	FR-09
POST	/api/admin/recruiters	Admin	Onboard a recruiter and company	FR-10, BR-05
PATCH	/api/admin/listings/:id/moderation	Admin	Approve, edit or remove a listing	FR-10
GET	/api/admin/analytics	Admin	Dashboard statistics	FR-11
GET	/api/prep/tests	Student	List mock tests	FR-12
POST	/api/prep/tests/:id/submit	Student	Submit answers and receive score	FR-12
GET	/api/prep/interview	Student	Interview preparation content	FR-12
GET	/api/notifications	Student, Recruiter	List own notifications	UC-11
GET	/api/health	Public	Service health check	NFR-04

Component interface 1: Authentication (Account and Profile Service)

Item	Definition
Endpoint	POST /api/auth/register
Request	{name: string (required), usn: string (required, unique), email: string (required, unique), password: string (required, minimum 8 characters with a letter and a digit), branch: string, semester: number}
Response 201	{success: true, data: {userId, role: 'Student'}}
Errors	400 VALIDATION_ERROR (missing or invalid field), 409 EMAIL_EXISTS or USN_EXISTS, 429 TOO_MANY_REQUESTS
Endpoint	POST /api/auth/login
Request	{email: string, password: string}
Response 200	{success: true, data: {token, expiresIn, user: {id, name, role}}}
Errors	400 VALIDATION_ERROR, 401 INVALID_CREDENTIALS (same message for unknown email and wrong password), 429 TOO_MANY_REQUESTS after repeated failures
Endpoint	POST /api/auth/logout (Bearer token required)
Response 200	{success: true}. The token identifier is stored in the revoked token list until its expiry.
Errors	401 UNAUTHORIZED

Component interface 2: Applications (Application Service)

Item	Definition
Endpoint	POST /api/applications (role Student)
Request	{listingId: string (required)}. The profile and resume reference are read from the student record on the server.
Response 201	{success: true, data: {applicationId, listingId, status: 'Applied', appliedAt}}
Errors	400 PROFILE_INCOMPLETE, 401 UNAUTHORIZED, 403 FORBIDDEN (not a Student), 404 LISTING_NOT_FOUND (also returned for unpublished listings), 409 DUPLICATE_APPLICATION (BR-01), 410 DEADLINE_PASSED (BR-04)
Endpoint	PATCH /api/applications/:id/status (role Recruiter)
Request	{status: 'Shortlisted' | 'Rejected' | 'Selected'}
Response 200	{success: true, data: {applicationId, status, updatedAt}}
Errors	400 INVALID_STATUS, 401 UNAUTHORIZED, 403 FORBIDDEN (recruiter does not own the listing, BR-03), 404 APPLICATION_NOT_FOUND
Endpoint	DELETE /api/applications/:id (role Student, owner)
Response 204	No body

Item	Definition
Errors	401 UNAUTHORIZED, 403 FORBIDDEN, 404 APPLICATION_NOT_FOUND, 409 DECISION_MADE (status is no longer Applied)
Endpoint	GET /api/listings?keyword=&role;=&company;=&location;=&deadlineBefore;=&page;=&limit;=
Response 200	{success: true, data: {items: [{id, title, company, location, deadline}], page, limit, total}}
Errors	400 VALIDATION_ERROR (invalid filter or limit above 50), 401 UNAUTHORIZED

4.4 Error Handling, Logging and Monitoring

Standard error format. Every failed request returns the same JSON shape so that the client can handle errors in one place:

```
{success: false, error: {code: 'DUPLICATE_APPLICATION', message: 'You already have an active application for this listing.', details: []}}
```

HTTP status	Used for
400	Validation failure, malformed input, incomplete profile, invalid status value
401	Missing, invalid, expired or revoked token; wrong login credentials
403	Authenticated user without the required role or without ownership of the resource
404	Resource does not exist or is not visible to the caller
409	Conflict with a business rule (duplicate email, duplicate application, decision already made)
410	Listing deadline has passed
413 / 415	Uploaded file too large or of a disallowed type
429	Rate limit exceeded
500	Unexpected server error; generic message returned, details written to the server log only

- A central error handler catches errors from all controllers and async handlers; services throw typed application errors that carry the code and status.
- Client-side: the React front end shows field-level messages for 400 responses, redirects to login on 401, and shows a clear access-denied page on 403.
- **Logging.** Structured logs with timestamp, request id, route, status and duration. Passwords, tokens and resume contents are never logged (SR-01). Failed logins, access-denied events and all admin actions are written to the AuditLog collection (SR-06).
- **Monitoring.** GET /api/health reports server and database connectivity for uptime checks during college working hours (NFR-04). Metrics reviewed during testing: error rate per route, slow request count against the 3 second target (NFR-01), and email send failures.

4.5 UX Design

- Role-specific dashboards after login: Student (search, applications, preparation), Recruiter (my listings, applicants), Admin (onboarding, moderation, analytics).
- Responsive layout for desktop and mobile viewports, consistent navigation and a shared component library for forms, tables and status badges (NFR-02).
- Inline validation messages and clear status badges (Applied, Shortlisted, Rejected, Selected) with distinct colours and text labels so that meaning does not depend on colour alone.
- Colour contrast chosen to meet WCAG AA contrast ratios; all form fields have labels.
- Supported browsers: latest two major versions of Chrome, Edge and Firefox (NFR-07).

4.6 Open Issues and Next Steps

- Maximum resume size is set to 2 MB and file type to PDF for the first release; the TPO may request a change.

- Real-time delivery of in-app notifications is implemented by polling in the first release; WebSocket delivery is a possible later enhancement.
- Deadline reminder emails need a scheduled job; the scheduling approach (node-cron inside the server) is to be confirmed in Sprint 5.
- Future enhancements outside the current scope: ATS integration, multi-college deployment and native mobile clients.

5. Appendices

5.1 Glossary

TPO, MERN, JWT and SRN/USN are defined in the SRS glossary. Additional terms: **SPA**, a web application that loads once and updates views in the browser; **RBAC**, restricting actions by user role; **ODM**, a library mapping database documents to objects; **TTL index**, a MongoDB index that removes documents after a set time; **STRIDE**, a threat classification model; **MVC**, a pattern separating model, view and controller.

5.2 References

- CampusReady SRS First Edition, Team 12, UE24CS341A.
- SAD and Software Test Plan templates, Department of Computer Science and Engineering, PES University.
- IEEE 42010, Systems and software engineering: Architecture description.
- OWASP Application Security Verification Standard and OWASP Top 10.
- NIST SP 800-160, Systems Security Engineering.

5.3 Tools

Diagrams were drawn with Python and Matplotlib and can be reproduced in draw.io or PlantUML. API contracts are verified with Postman. Source control on GitHub, CI/CD with Jenkins, containerisation with Docker.

PES UNIVERSITY, BENGALURU

Department of Computer Science and Engineering

UE24CS341A, Software Engineering | Level 3 Mini-Project

SOFTWARE TEST PLAN

(STP) Version 1.0

CampusReady: Online Internship and Placement Preparation Portal

Project	CampusReady: Online Internship and Placement Preparation Portal
Team Number	Team 12
Version	1.0
Authors	Sejal Siddapuram, Sampath D Naik, Sagar A Walikar, Saiesh Shankar
Date	01-10-2026
Status	Draft

Team Members

Sl.	Student Name	USN	Module Owned
1	Sejal Siddapuram	PES1UG24CS426	Account and Profile Management (FR-01 to FR-03)
2	Sampath D Naik	PES1UG24CS416	Listings and Applications (FR-04 to FR-06)
3	Sagar A Walikar	PES1UG24CS396	Recruiter Portal (FR-07 to FR-09)
4	Saiesh Shankar	PES1UG24CS401	Admin and Preparation (FR-10 to FR-12)

1. Introduction

Purpose. This document defines the test plan for CampusReady Version 1.0. It states the objectives, scope, strategy, environment, schedule, responsibilities and traceability for testing, and it includes the test cases that cover the functional, non-functional, security and business rule requirements of the SRS.

Scope. Testing covers student registration, login and profile management, listing search, application submission and tracking, the recruiter portal, the admin and TPO dashboard, the mock test module, notifications, and the non-functional and security requirements of the SRS. Hosting infrastructure and third-party service internals are excluded (Section 4).

References. CampusReady SRS First Edition; CampusReady Software Architecture and Design Specification Version 1.0; STP template of the Department of Computer Science and Engineering, PES University.

Definitions. FR (functional requirement), NFR (non-functional requirement), SR (security requirement), BR (business rule), TC (test case), RTM (requirements traceability matrix), UAT (user acceptance test), RBAC (role-based access control), JWT (JSON Web Token).

2. Test Items

- Account and Profile module: registration, login, logout, token handling, profile and resume upload (Sejal Siddapuram)
- Listings and Applications module: search and filter, apply, withdraw, status tracking (Sampath D Naik)
- Recruiter Portal module: listing management, applicant review, status updates, summary counts (Sagar A Walikar)
- Admin and Preparation module: onboarding, moderation, analytics, mock tests, interview content (Saiesh Shankar)
- Shared components: authentication and RBAC middleware, validation, error handler, audit logger, Notification Service, REST API as a whole, React user interfaces

3. Features to be Tested

Features are mapped to the requirement identifiers of the SRS.

Req ID	Feature	Priority
FR-01	Student registration with college credentials	High
FR-02	Login and role-based access control	High
FR-03	Student profile editing and resume upload	High
FR-04	Listing search and filter by keyword, role, company, location and deadline	High
FR-05	Application submission with profile and resume	High
FR-06	Application status display, withdrawal and status change notification	High
FR-07	Recruiter creates, edits and publishes listings	High
FR-08	Recruiter views applicants and updates status	High
FR-09	Applicant count by status per listing	Medium
FR-10	Admin onboarding of recruiters and moderation of postings	High
FR-11	Admin dashboard statistics	Medium
FR-12	Mock aptitude test with automatic scoring and interview content	Medium
NFR-01	Core pages load within 3 seconds under normal load	High
NFR-02	Usability for a first-time student on desktop and mobile	Medium
NFR-03	Application status transitions persist without data loss	High
NFR-04	Availability during college working hours (best effort)	Low

Req ID	Feature	Priority
NFR-05	Modular MVC-style structure	Medium (reviewed, not executed)
NFR-06	Growth to single-college data volume without redesign	Medium
NFR-07	Rendering on latest two versions of Chrome, Edge and Firefox	Medium
SR-01 to SR-06	Password hashing, HTTPS, server-side RBAC, token expiry and invalidation, upload validation, security event logging	High
BR-01 to BR-05	One active application per listing; only approved listings visible; only owning recruiter changes status; no applications after deadline; only Admin onboards recruiters	High

4. Features Not to be Tested

- Internal behaviour of MongoDB, Node.js, the React library and other third-party libraries (assumed tested by their maintainers).
- Delivery of email by the external SMTP provider beyond the application handing the message over (the SMTP service is stubbed).
- Hosting provider infrastructure, network and operating system configuration.
- Items declared out of scope in the SRS: payment processing, native mobile applications, video interview proctoring, third-party ATS integration and multi-tenant deployment.
- NFR-05 (modular structure) is verified by code review and static analysis, not by test execution.

5. Test Approach / Strategy

Levels

- **Unit tests:** services and utility functions with mocked models, using Jest. Target of at least 70 percent line coverage on the service layer, with branch coverage reported. Written and run by each module owner.
- **Integration tests:** REST endpoints against a test MongoDB instance using Jest and Supertest, covering middleware, controllers, services and models together; SMTP stubbed.
- **System tests:** end-to-end flows through the React interface for each role, using Cypress for automated flows and manual execution for the rest.
- **Acceptance tests:** demonstration of the main use cases (UC-01 to UC-11) to the instructor and, where possible, a TPO representative at the end of Sprint 6.

Types

- Functional testing of every FR and BR through positive, negative and boundary cases.
- Regression testing: the automated unit and integration suites run on every push and pull request through the Jenkins pipeline.
- Performance testing with k6 or JMeter for NFR-01 and NFR-06.
- Usability testing by task-based walkthrough with first-time users (NFR-02).
- Compatibility testing across supported browsers (NFR-07).
- Static analysis with SonarQube or a similar tool, and code review of every pull request.

Entry criteria. A stable build is deployed to the test environment, test data is seeded, the API contract of the module under test is frozen for the sprint, and the module's unit tests pass in the pipeline.

Exit criteria. All planned test cases are executed, with actual result and pass or fail recorded; no critical or high severity defect remains open; every FR, NFR, SR and BR has at least one executed test case or review record; the coverage report and security validation results are attached.

5.1 Security Validation

Security validation is planned against SR-01 to SR-06 and the threats in the STRIDE model of the SAD, Section 3.9.

Activity	Method	Requirement	Test case
Password storage	Inspect stored documents and logs for plaintext; confirm bcrypt hash format	SR-01	TC-SEC-01
Transport security	Check HTTP to HTTPS redirect, TLS version and HSTS header with a TLS scanner	SR-02	TC-SEC-02
Authorization	Call every protected route with each wrong role and with no token, directly against the API; build a route versus role matrix and verify every cell	SR-03	TC-SEC-03
Token handling	Expired token, token after logout, tampered token signature	SR-04	TC-SEC-04
Upload validation	Wrong type, disguised extension, oversized file, valid file	SR-05	TC-SEC-05
Audit logging	Trigger failed logins and admin actions; confirm log entries; confirm no secrets in logs	SR-06	TC-SEC-06
Injection and XSS	NoSQL operator injection in login and search; script payloads in text fields; fuzzing of input fields with malformed and oversized values	SR-03, SAD 3.9	TC-SEC-07
Brute force	Repeated login attempts must trigger rate limiting (429)	SAD 3.9	TC-SEC-06
Automated scans	OWASP ZAP baseline scan of the deployed test environment; npm audit for vulnerable dependencies; SonarQube security hotspots review	SR-01 to SR-06	Scan reports
Penetration testing	Manual attempts at authentication bypass, privilege escalation between Student, Recruiter and Admin, and access to another user's resume	SR-03, SR-04	TC-SEC-03, TC-SEC-04

6. Test Environment

Item	Detail
Hardware	Developer laptops and one test server or cloud virtual machine
Software	CampusReady server (Node.js and Express.js), React front end, MongoDB test database, Docker containers, SMTP stub or sandbox
Browsers	Latest two major versions of Chrome, Edge and Firefox; desktop and mobile viewports
Tools	Jest, Supertest, React Testing Library, Cypress, Postman, k6 or JMeter, OWASP ZAP, SonarQube or similar, Jenkins, Git and GitHub, Jira or GitHub Issues for defects
Test data	Seeded accounts for Student, Recruiter and Admin; Draft, Published, approved and unapproved listings; listings before, on and after the deadline; applications in every status; mock test question bank; large data set for NFR-06

7. Test Schedule

Testing follows the six-sprint plan of the SRS. Each module owner writes test cases before or with the code and executes them in the sprint in which the feature is delivered.

Sprint	Testing activity	Requirements covered
Sprint 1 to 2	Test environment and pipeline setup; unit and integration tests for account, login, profile and search	FR-01 to FR-04, SR-01, SR-04
Sprint 3	Application submission, withdrawal and status tests; business rule tests	FR-05, FR-06, BR-01, BR-04
Sprint 4	Recruiter portal tests; ownership and RBAC tests	FR-07 to FR-09, BR-03
Sprint 5	Admin, mock test and notification tests; security validation and scans	FR-10 to FR-12, SR-01 to SR-06, BR-02, BR-05
Sprint 6	Non-functional tests, full regression, system tests, defect closure, UAT and test summary report	NFR-01 to NFR-07, all

8. Test Deliverables

- Test Plan (this document)

- Test cases (Section 13 and the test case table in this plan; automated tests in the repository)
- Test scripts: Jest, Supertest, Cypress, k6 or JMeter, Postman collection
- Test data and seed scripts
- Test execution logs and CI pipeline results
- Unit test and coverage report (line and branch)
- Security validation results and scan reports
- Defect reports
- Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
Test Engineer and Developer, Account and Profile	Sejal Siddapuram	Design and execute tests for FR-01 to FR-03 and SR-01, SR-04, SR-05; fix defects in the module
Test Engineer and Developer, Listings and Applications	Sampath D Naik	Design and execute tests for FR-04 to FR-06, BR-01, BR-04
Test Engineer and Developer, Recruiter Portal	Sagar A Walikar	Design and execute tests for FR-07 to FR-09, BR-03
Test Engineer and Developer, Admin and Preparation	Saiesh Shankar	Design and execute tests for FR-10 to FR-12, BR-05, SR-06
Cross-review	All members	Each member reviews the tests of one other module; NFR and security tests are shared and assigned per sprint
Approver	Course instructor / TPO representative	Review results and approve readiness

10. Risks and Mitigation

Risk	Mitigation
Delay in stable build for a sprint	Daily integration to the main branch; early smoke tests in the pipeline
Test environment unavailable	Docker-based environment that every member can start locally
Dependency on SMTP service	Stub the SMTP service in automated tests; use a sandbox account for manual checks
Small team, tests written late	Test cases are written in the same sprint as the feature; pull requests need tests
Performance tests not representative	Define normal load as 50 concurrent users and seed realistic data volumes

11. Assumptions and Dependencies

- The API contracts of the SAD, Section 4.3, are stable within a sprint.
- Seed data and test accounts are available before test execution starts.
- Normal load for NFR-01 is taken as 50 concurrent users on a broadband connection, since the SRS does not define a number; the TPO may revise it.
- The test environment has network access to an SMTP sandbox or stub.

12. Suspension and Resumption Criteria

Suspend testing if: the environment is unavailable for more than 4 hours; the build is so unstable that more than 30 percent of planned test cases are blocked; or a critical security defect is found, until it is fixed.

Resume testing when: blocking defects are resolved and verified, and the environment is stable.

13. Test Case Management and Traceability

Test cases are managed in this plan and in the repository test folders. Each test case carries a unique identifier, the requirement it verifies, its level and type, the steps and data, the expected result, the actual result and the pass or fail status. The actual result and pass or fail columns are filled after execution in each sprint and in the Test Summary Report.

13.1 Test Cases

Test Case ID	Req ID	Level / Type	Description, steps and data	Expected result	Actual result	Pass / Fail
TC-AUTH-01	FR-01	Unit / Integration	Register with valid name, USN, email and password (8+ chars, letter and digit).	201; user created with role Student; StudentProfile linked.	To be filled	Pending
TC-AUTH-02	FR-01	Integration	Register with an email already in use.	409 EMAIL_EXISTS; no second record created.	To be filled	Pending
TC-AUTH-03	FR-02	Integration	Login with correct email and password.	200; JWT returned with role and expiry.	To be filled	Pending
TC-AUTH-04	FR-02	Integration	Login with a correct email and a wrong password, then with an unknown email.	401 INVALID_CREDENTIALS for both, identical message.	To be filled	Pending
TC-PROF-01	FR-03	System	Student edits branch, semester, skills and uploads a valid 1 MB PDF resume.	200; profile saved; resume reference stored; values visible on reload.	To be filled	Pending
TC-LIST-01	FR-04, BR-02	Integration	Search with keyword and location filters while Draft, Published and unapproved listings exist.	Only matching Published and approved listings returned, paginated.	To be filled	Pending
TC-APP-01	FR-05	System	Student with complete profile applies to a Published listing before its deadline.	201; application with status Applied; visible to student and owning recruiter.	To be filled	Pending
TC-APP-02	FR-05, BR-01	Integration	Apply twice to the same listing.	Second request returns 409 DUPLICATE_APPLICATION; one record only.	To be filled	Pending
TC-APP-03	BR-04	Unit / Integration	Apply to a listing whose deadline is yesterday (boundary: deadline is today).	Past deadline: 410 DEADLINE_PASSED. On the deadline day: accepted.	To be filled	Pending
TC-APP-04	FR-06, BR-01	Integration	Withdraw an application with status Applied, then apply again; try withdrawing one with status Shortlisted.	First withdrawal 204 and re-application allowed; withdrawal after decision 409 DECISION_MADE.	To be filled	Pending
TC-NOT-01	FR-06, UC-11	Integration	Recruiter changes an application status; SMTP service is stubbed once as available and once as failing.	In-app notification stored in both cases; email sent when available; failure logged and request still succeeds.	To be filled	Pending
TC-REC-01	FR-07	System	Recruiter creates a listing with title, description, eligibility and deadline, then publishes it.	201 Draft, then Published; missing mandatory field gives 400.	To be filled	Pending
TC-REC-02	FR-08, BR-03	Integration	Recruiter B updates the status of an application on Recruiter A's listing.	403 FORBIDDEN; status unchanged.	To be filled	Pending
TC-REC-03	FR-08	System	Owning recruiter views applicants and sets Shortlisted, then Selected.	Statuses saved and shown to the student.	To be filled	Pending
TC-REC-04	FR-09	Unit	Listing with 2 Applied, 1 Shortlisted, 1 Rejected, 1 Selected applications requests its summary.	Counts returned: Applied 2, Shortlisted 1, Rejected 1, Selected 1.	To be filled	Pending

Test Case ID	Req ID	Level / Type	Description, steps and data	Expected result	Actual result	Pass / Fail
TC-ADM-01	FR-10, BR-05	Integration	Admin onboards a recruiter; the same request is sent with a Student token and a Recruiter token.	Admin: 201. Student and Recruiter: 403 FORBIDDEN.	To be filled	Pending
TC-ADM-02	FR-10, BR-02	System	Admin approves one listing and removes another.	Approved listing visible to students; removed listing no longer visible; both actions in AuditLog.	To be filled	Pending
TC-ADM-03	FR-11	Integration	Admin opens analytics with known seeded data.	Totals of students, listings, applications and selections match the seed data.	To be filled	Pending
TC-PREP-01	FR-12	Unit / System	Student submits a mock test with 7 of 10 answers correct.	Score 7 of 10 stored as TestAttempt and displayed.	To be filled	Pending
TC-SEC-01	SR-01	Security	Inspect stored user document and server logs after registration and login.	passwordHash is a bcrypt hash; no plaintext password in database or logs.	To be filled	Pending
TC-SEC-02	SR-02	Security	Request the site over plain HTTP and run a TLS scan on the HTTPS endpoint.	HTTP redirected to HTTPS; TLS 1.2 or higher only; HSTS header present.	To be filled	Pending
TC-SEC-03	SR-03, FR-02	Security	Call every Recruiter and Admin route using a Student token, and Admin routes using a Recruiter token, bypassing the UI.	403 FORBIDDEN on every call; no data returned.	To be filled	Pending
TC-SEC-04	SR-04	Security	Use an expired token, and a token after logout.	401 UNAUTHORIZED in both cases.	To be filled	Pending
TC-SEC-05	SR-05	Security	Upload an .exe renamed to .pdf, a 5 MB PDF and a valid PDF.	Disguised file and oversized file rejected (415 / 413); valid file accepted.	To be filled	Pending
TC-SEC-06	SR-06	Security	Make 5 failed logins and perform one admin action.	Failed logins and the admin action appear in AuditLog with user, time and source address.	To be filled	Pending
TC-SEC-07	SR-03, Security	Security	Send login body {email: {"$ne": null}, password: {"$ne": null}} and script tags in profile and listing fields.	Request rejected or sanitised; no authentication bypass; scripts rendered as inert text.	To be filled	Pending
TC-NFR-01	NFR-01	Performance	50 concurrent users run listing search and dashboard load for 5 minutes (normal load).	95th percentile page load of 3 seconds or less.	To be filled	Pending
TC-NFR-02	NFR-02	Usability	Five first-time students find a listing and apply, with no training, on desktop and mobile viewports.	All five complete the task unaided; issues recorded.	To be filled	Pending
TC-NFR-03	NFR-03	Reliability	Change several application statuses, restart the server, read the records.	All statuses persisted exactly as set.	To be filled	Pending
TC-NFR-04	NFR-04	Operational	Poll GET /api/health every minute during college working hours across the test period.	Uptime figure recorded; outages explained (best effort target).	To be filled	Pending
TC-NFR-05	NFR-06	Performance	Seed 1,000 listings and 10,000 applications, then repeat the search test.	Search results within 3 seconds; indexes used.	To be filled	Pending

Test Case ID	Req ID	Level / Type	Description, steps and data	Expected result	Actual result	Pass / Fail
TC-NFR-06	NFR-07	Compatibility	Run the main student, recruiter and admin flows on the latest two versions of Chrome, Edge and Firefox.	Layout and functions work with no blocking defect.	To be filled	Pending

Total test cases: 32, covering all 12 functional requirements, all 6 security requirements, 5 of the 7 non-functional requirements through execution (NFR-05 by review), and all 5 business rules.

13.2 Requirements Traceability Matrix (SRS to Architecture and Tests)

Req ID	Architecture component (SAD)	Test case IDs
FR-01	Account and Profile Service; Sequence Diagram 1	TC-AUTH-01, TC-AUTH-02
FR-02	Auth and RBAC Middleware; Account Service	TC-AUTH-03, TC-AUTH-04, TC-SEC-03
FR-03	Account and Profile Service; File Storage	TC-PROF-01, TC-SEC-05
FR-04	Listing Service	TC-LIST-01
FR-05	Application Service; Sequence Diagram 2	TC-APP-01, TC-APP-02, TC-APP-03
FR-06	Application Service; Notification Service	TC-APP-04, TC-NOT-01
FR-07	Listing Service	TC-REC-01
FR-08	Recruiter Service; Sequence Diagram 3	TC-REC-02, TC-REC-03
FR-09	Recruiter Service	TC-REC-04
FR-10	Admin Service	TC-ADM-01, TC-ADM-02
FR-11	Admin Service	TC-ADM-03
FR-12	Preparation Service	TC-PREP-01
NFR-01	API layer, MongoDB indexes, pagination	TC-NFR-01
NFR-02	React user interfaces	TC-NFR-02
NFR-03	Application Model, atomic updates	TC-NFR-03
NFR-04	Health endpoint, deployment	TC-NFR-04
NFR-05	Layered structure (SAD 3.5)	Code review and SonarQube report
NFR-06	Indexes, pagination	TC-NFR-05
NFR-07	React user interfaces	TC-NFR-06
SR-01	Account Service (bcrypt)	TC-SEC-01
SR-02	HTTPS, helmet	TC-SEC-02
SR-03	RBAC Middleware	TC-SEC-03, TC-SEC-07, TC-REC-02, TC-ADM-01
SR-04	Auth Middleware, RevokedToken	TC-SEC-04
SR-05	File upload validation	TC-SEC-05
SR-06	Audit Logger	TC-SEC-06, TC-ADM-02
BR-01	Application Service	TC-APP-02, TC-APP-04
BR-02	Listing Service, Admin Service	TC-LIST-01, TC-ADM-02
BR-03	Application and Recruiter Service	TC-REC-02
BR-04	Application Service	TC-APP-03
BR-05	Admin Service, RBAC Middleware	TC-ADM-01

14. Test Metrics and Reporting

Metrics collected: percentage of test cases executed; percentage passed and failed; defect density per module; defect aging; requirement coverage; unit test line and branch coverage; performance results against

NFR-01.

Reports: test execution status at the end of each sprint, coverage and CI reports from the pipeline, security validation results, and the final Test Summary Report.

15. Approvals

Role	Name	Signature / Date
Team Representative		
Course Instructor		
TPO Representative (if available)		